

# ARGUS Project Proposal

Revision 2 — Senior Design 2026

## ARGUS — Revised Project Proposal

Autonomous Residential Guardian & Utility System Senior Design, 2026 · Revision 2 · 2026-09-01

### Revision note

This supersedes the original proposal. It exists because the first version described a system we did not end up building, and claimed performance we have since measured and found to be wrong. Both are corrected below and marked ▲.

| Area               | Original proposal                    | Revision 2                                           |
|--------------------|--------------------------------------|------------------------------------------------------|
| Object model       | MobileNet-SSD, TensorFlow Lite       | YOLOv8n via Ultralytics ▲                            |
| Face model         | FaceNet + MediaPipe                  | ArcFace ( <code>buffalo_l</code> ) via InsightFace ▲ |
| Accuracy claim     | ">95% accuracy, <5% false positives" | Measured; see §5. The claim was unsupported ▲        |
| Threat detection   | Not in scope                         | Third model, integrated and measured ▲               |
| Federated learning | Mentioned as future work             | The project's central contribution ▲                 |
| Licensing          | "MIT License"                        | AGPL-3.0, and required ▲                             |

### 1. Problem

Home and small-site security cameras present their owner with a choice they should not have to make.

**Cloud systems** send continuous footage of private space to a vendor's servers. They work well, they improve over time because the vendor aggregates everyone's video, and they charge a subscription indefinitely. The price of the intelligence is the footage.

**Local systems** keep the footage private and cost nothing per month. They also never improve: the model shipped on the device is the model it has forever, and it was trained on scenes that look nothing like the customer's front door.

The trade is treated as fundamental. It is not.

## 2. Proposal

---

ARGUS is an edge security system where **all video is processed and stored on the customer's own device**, and where **the detection model still improves over time** by learning across every deployment — without any footage being transmitted.

The mechanism is federated learning. Each device trains on what its own camera saw; every two weeks it sends only the resulting *model changes* — arrays of numbers — to a central server, which combines them into an improved shared model and sends it back.

The research question this raises, and the one the project actually answers, is the one that makes federated learning hard in practice:

If devices we do not control contribute to a shared model, how do we stop one broken or malicious device from degrading it for everyone?

---

## 3. Objectives

---

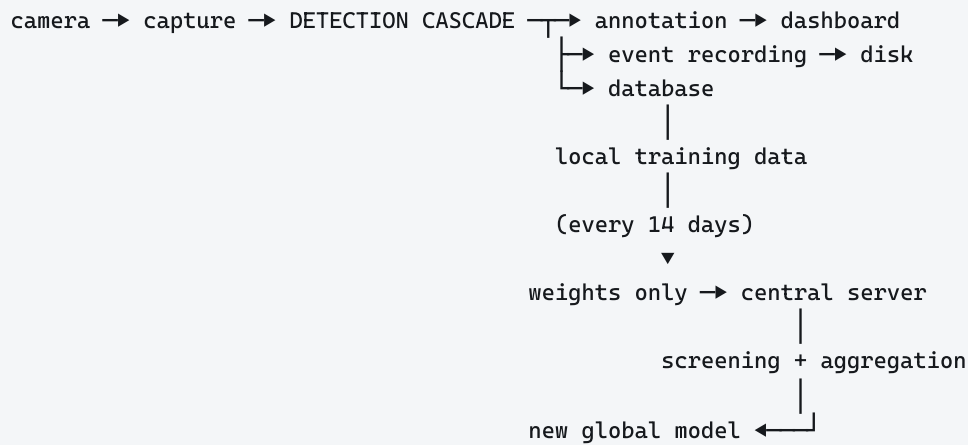
| #  | Objective                                     | Measure                       | Status           |
|----|-----------------------------------------------|-------------------------------|------------------|
| O1 | Multi-class detection on-device               | 7 classes, measured stability | Complete         |
| O2 | Face recognition, enrol and match live        | Verified on hardware          | Complete         |
| O3 | Threat detection as a distinct stage          | Measured against ground truth | Complete         |
| O4 | Footage never leaves the device               | Structural, not policy        | Complete         |
| O5 | Federated learning on a 14-day cycle          | Survives reboots              | Complete         |
| O6 | Defend the global model from bad contributors | Adversarial tests             | Complete         |
| O7 | Operator interface                            | Live dashboard                | Complete         |
| O8 | Authenticated remote access                   | Tunnel, auth enforced         | Complete         |
| O9 | 10–15 FPS on target hardware                  | Benchmark on the Pi           | Pending hardware |

---

## 4. System design

---

Nine subsystems; full plain-language treatment in [SUBSYSTEMS.md](#).



## 4.1 The cascade

Cheap tests gate expensive ones, so a still scene costs almost nothing:

```

motion (frame differencing)
├── YOLOv8n objects → HUMAN / VEHICLE / ANIMAL / UNKNOWN
│   └── if HUMAN: ArcFace identity → FACE
└── YOLO11n threat → DANGEROUS_PERSON
  
```

The threat stage runs on the whole frame rather than on cropped people. This was measured, not assumed: cropping to detected people lost **31% of recall**, because 20% of armed people were never found by the object model in the first place, so the crop stage never ran on them.

## 4.2 Hardware

| Component   | Part                                      | Role                      |
|-------------|-------------------------------------------|---------------------------|
| Compute     | Raspberry Pi 5 (8 GB)                     | One per camera node       |
| Accelerator | AI HAT+ 2 (Hailo-10H, 40 TOPS INT8, 8 GB) | Runs both YOLO models     |
| Camera      | Camera Module 3 (IMX708)                  | CSI                       |
| Server      | Any always-on host                        | Global model, aggregation |

One Pi per camera. This is a deliberate change from an earlier single-host design, and it has a property worth stating: **each camera node is an independent federated-learning contributor**, so three cameras is three contributors — the threshold at which the defence in §6 becomes fully effective. A single host with three cameras is one contributor no matter how many cameras it has.

## 5. Measured performance ▲

The original proposal claimed ">95% detection accuracy with <5% false positive rate." That figure was not measured and is not achievable with this class of model. Real numbers, against the 314-image labelled test set and 128 ordinary photographs as negatives:

### Face recognition

| Measure                                 | Result           |
|-----------------------------------------|------------------|
| Same person, clean stills               | 0.986 similarity |
| Different person, best impostor of five | 0.073            |
| Separation                              | 0.913            |
| False accepts                           | 0 of 5           |

This is the strongest component in the system.

### Threat detection

| Threshold               | Precision | Recall | False alarms per 100 ordinary scenes |
|-------------------------|-----------|--------|--------------------------------------|
| 0.35 (upstream default) | 0.901     | 0.800  | 29.7                                 |
| 0.55 (ours)             | 0.919     | 0.768  | 18.8                                 |
| 0.70                    | 0.931     | 0.700  | 10.2                                 |

We tuned to 0.55. F1 peaks nearer 0.30, but F1 is the wrong objective here: at the default, nearly one ordinary scene in three flags somebody as armed. Pointed at a visitor walking past a booth, that is both a credibility problem and an ethical one.

**Stated plainly: recall 0.768 means roughly one armed person in four is missed, and about 19 ordinary scenes in 100 raise a false flag.** This is a review aid, not a guard. Any claim stronger than that is unsupportable.

### Object classes

Stability matters more than peak confidence — a box near the decision threshold flickers on and off between frames and reads as broken.

| Class            | Detections near threshold | Verdict              |
|------------------|---------------------------|----------------------|
| VEHICLE          | 6%                        | Most stable          |
| DANGEROUS_PERSON | 4%                        | Stable               |
| HUMAN            | 12–18%                    | Stable               |
| ANIMAL           | 18%                       | Stable, small sample |
| UNKNOWN          | 24%                       | Noise; see §8        |

## 6. Contribution: defending a federated model ▲

Standard federated averaging (FedAvg) combines every update it receives. One device that is broken, diverged, or hostile therefore moves the shared model — and the damaged model is distributed to every other device, so a single bad contributor degrades the entire fleet and the next round begins from the damage.

Our aggregator screens updates through four filters, cheapest first:

| # | Filter       | Catches                                                    |
|---|--------------|------------------------------------------------------------|
| 1 | Structural   | Malformed shapes, NaN, Inf, false sample counts            |
| 2 | Magnitude    | Updates far larger than peers — model-replacement attacks  |
| 3 | Direction    | Updates opposing consensus — sign-flip, targeted poisoning |
| 4 | Trimmed mean | Residual per-coordinate outliers                           |

Followed by a **validation gate**: the candidate model is scored, and if it is worse than the model it would replace, it is discarded and the incumbent kept.

Two design decisions worth defending:

- **Screening operates on model *changes*, not model weights.** Raw weights are dominated by the shared starting point, so two updates that disagree completely still appear nearly identical. Comparing changes is what makes disagreement visible at all.
- **Consensus uses the median, not the mean.** The mean is precisely what an attacker manipulates. A median of  $n$  values tolerates up to  $(n-1)/2$  arbitrary corruptions; a mean tolerates none.

### Stated limitation

Filters 3 and 4 compare contributors against each other, so they require a majority — **at least three contributors**. With two devices that disagree, there is no principled way to determine which is wrong. The system detects this condition, reports it, and falls back to filters 1 and 2 plus the validation gate, which are absolute rather than comparative.

Validation: 21 adversarial tests, each implementing an attack and asserting the defence holds, including a counterfactual demonstrating that the sign-flip attack succeeds against plain averaging.

---

## 7. Privacy

---

| Leaves the device           | Never leaves the device |
|-----------------------------|-------------------------|
| Model weight arrays         | Camera frames           |
| A count of training samples | Recorded clips          |
| Loss and accuracy numbers   | Face images             |
| A device identifier         | Face embeddings         |

**Face identity never participates in federated learning at all.** Face embeddings are stored in each device's local database and used only for matching there; they are not part of the model that federates, so they cannot appear in any update.

**One honest clarification:** federated learning means data does not *leave* the device. It does not mean data is not *stored*. Each device keeps small training crops and event clips locally, on hardware the customer owns. "We never store your footage" would be false; "your footage never leaves your device" is true.

---

## 8. Scope: what is not built

---

Named deliberately, because a proposal that lists only successes is not useful for evaluation.

**Security** - No cryptographic signing of updates. A device is identified by a string it chooses; the defence is against bad *updates*, not forged identities. - No secure aggregation. The server sees each device's update individually. - No differential privacy. No formal guarantee, only the structural one. - Encryption at rest is a stub. Face embeddings are stored unencrypted.

**Detection** - **UNKNOWN** covers 69 unmapped object classes and produces visual noise. Filtering it is a one-line change but a product decision. - No night vision. No component in the system addresses low light, and the cameras have no IR illumination.

**Deployment** - Robust aggregation is complete and tested but **not yet connected to the Flower server**, which still runs standard FedAvg. - No Hailo hardware has been run against this code. The compilation path and probe logic are implemented and tested; end-to-end frame rate on the accelerator is unverified.

---

## 9. Licensing ▲

---

The original proposal stated MIT. That was not accurate and not available.

ARGUS links Ultralytics (AGPL-3.0) and ships a third-party threat-detection model that is also AGPL-3.0. AGPL's network clause applies because ARGUS serves video over a network. The project is therefore **AGPL-3.0**, with third-party components and dataset attribution recorded in `NOTICE.md`.

The repository previously had no licence file at all, only a line in the README. That gap is closed.

---

## 10. Verification

| Area                             | Tests |
|----------------------------------|-------|
| Detection cascade                | 11    |
| Threat stage                     | 30    |
| Face recognition + API           | 35    |
| Authentication                   | 33    |
| Robust aggregation (adversarial) | 21    |
| Event recording                  | 18    |
| Federated scheduling             | 23    |
| Platform + multi-camera          | 35    |
| Dashboard, streaming, other      | 43    |
| Total                            | 249   |

All passing. Tests are the largest single body of code in the project (2,959 lines against 1,960 for the detection subsystem), which is deliberate: the failures that matter on a demo day are the silent ones — the ones where the system reports success and detects nothing.

---

# 11. Remaining work

| Item                                                       | Blocking?                          |
|------------------------------------------------------------|------------------------------------|
| Benchmark on Pi + Hailo hardware                           | Yes — O9 is unverified             |
| Connect robust aggregation to the Flower server            | Yes — the contribution is not live |
| Compile both models for Hailo (needs an x86-64 Linux host) | Yes                                |
| Three-node federated run                                   | Demonstrates the full defence      |
| Threshold tuning under venue lighting                      | Recommended                        |
| Night-vision decision                                      | Scope call                         |